

```
$ ./init_presentation --mode=technical
```

DFS: Sistema de Arquivos Distribuído

Arquitetura em camadas, sharding determinístico e comunicação TCP

```
USERS:      $Higor Ferreira - 202201635  
            $Vitória Mendonça - 202004699  
  
HOST:       dfs-marco-2  
  
STATUS:     System Operational
```

OBJETIVOS DO MARCO 2

Os três pilares definidos na especificação do trabalho.

[COMPLETED] 01. DISTRIBUIÇÃO DE DADOS

Particionamento de arquivos em chunks e roteamento determinístico entre múltiplos nós de armazenamento.

[COMPLETED] 02. BALANCEAMENTO INICIAL

Distribuição uniforme dos chunks via função de hash, sem hotspots ou tabelas centrais de roteamento.

[COMPLETED] 03. COMUNICAÇÃO ENTRE NÓS

Protocolo TCP com serialização Protobuf e framing por tamanho, atendendo múltiplos clientes em threads dedicadas.

ARQUITETURA EM CAMADAS

Quatro camadas com responsabilidades isoladas, cada uma resolvendo um problema específico do sistema distribuído.

[01] COORDENADOR CÉREBRO DO SISTEMA

Ponto de entrada do sistema. Calcula o destino de cada chunk via sharding e roteia as operações ao nó correto. Atende múltiplos clientes em threads dedicadas.

[02] NÓS DE ARMAZENAMENTO PERSISTÊNCIA FÍSICA

Processos independentes, cada um com sua porta TCP e diretório próprio em disco. Executam apenas operações locais sem participar de decisões de roteamento.

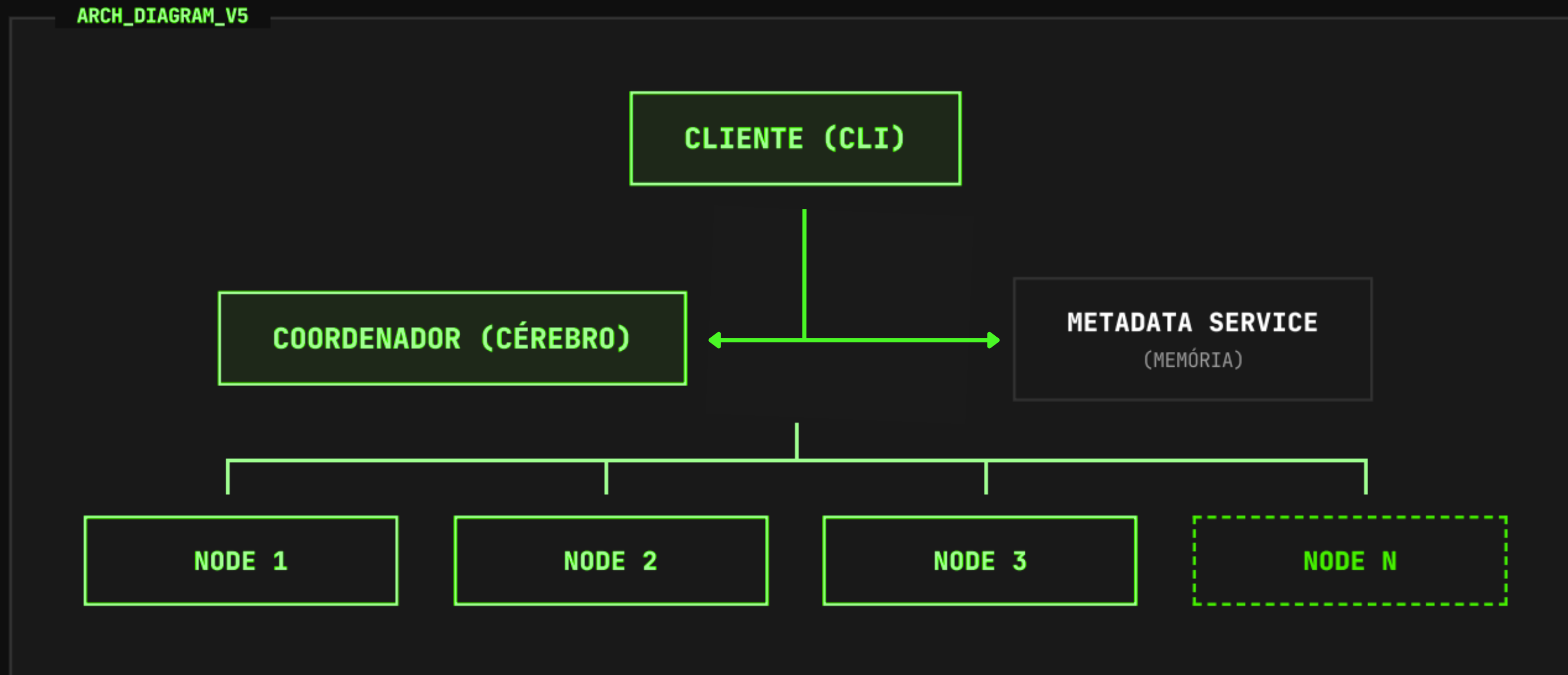
[03] METADADOS ÍNDICE DE ESTADO

Índice persistente em JSON que registra a localização dos chunks de cada arquivo. Consultado pelo coordenador nas operações de leitura, remoção e listagem.

[04] COMUNICAÇÃO PROTOCOLO DE REDE

TCP puro entre todos os processos, com mensagens serializadas em Protobuf e enquadradas por um cabeçalho de 4 bytes. 0 framing resolve o problema do stream TCP.

VISÃO GERAL DA ARQUITETURA



O cliente nunca interage diretamente com os nós. O Coordenador é o único ponto de contato externo, consultando o Metadata Service para mapear arquivos lógicos aos chunks distribuídos pelo cluster.

PROTOCOLO DE COMUNICAÇÃO

Três camadas resolvendo três problemas distintos da rede distribuída.

[L1] TCP

Transporte confiável e orientado a conexão. Garante entrega em ordem, mas trata os dados como fluxo contínuo de bytes (stream).

[L2] PROTOBUF

Serialização compacta e tipada de mensagens (FileRequest, FileResponse). Substitui parsing manual e independe de plataforma.

[L3] FRAMING

Cabeçalho de 4 bytes com o tamanho do payload. Resolve o problema clássico de "onde uma mensagem termina e a próxima começa" no fluxo TCP.

EXEMPLO DE ENQUADRAMENTO (FRAMING)

4 BYTES
(tamanho)

PAYLOAD PROTOBUF (FileRequest serializado)

SHARDING: COMO O DESTINO É CALCULADO

ALGORITMO DE DISTRIBUIÇÃO DETERMINÍSTICA

$$\text{shard_id} = \text{SHA256}(\text{path} + "::" + \text{chunk_id})[:8] \bmod N$$

N = número de nós ativos no cluster

POR QUE SHA-256?

- > Distribuição uniforme (efeito avalanche)
- > Sem colisões nem hotspots em nós específicos
- > Independente do conteúdo do arquivo (usa o path)

POR QUE POR CHUNK?

- > Espalha pedaços de um mesmo arquivo entre nós
- > Permite paralelismo real em arquivos grandes
- > Granularidade fina para rebalanceamento futuro

Exemplo: "video.mp4" → chunk 0 → $\text{SHA256}(\text{"video.mp4::0"}) \bmod 3 = \text{node } 1$
chunk 1 → $\text{SHA256}(\text{"video.mp4::1"}) \bmod 3 = \text{node } 2$
chunk 2 → $\text{SHA256}(\text{"video.mp4::2"}) \bmod 3 = \text{node } 0$

FLUXO DO PUT: ESCRITA

01. CHUNKING

Arquivo dividido em pedaços de tamanho fixo (64 KB).

02. HASHING

Para cada chunk, o ShardingManager calcula o nó de destino via SHA-256.

03. ROTEAMENTO

Coordenador envia cada chunk via TCP ao nó correto, com fallback se o primário falhar.

04. METADATA

Após confirmação de todos os chunks, o índice JSON é atualizado.

WRITE_FLOW_V4

CLI

| PUT video.mp4



COORDENADOR

| [chunk_0] —▶ node 1

| [chunk_1] —▶ node 2

| [chunk_2] —▶ node 0



METADATA (JSON) — confirmação

FLUXO DO GET: RECONSTRUÇÃO DO ARQUIVO

O coordenador atua como agregador, consultando o índice de metadados para localizar e reunir todos os chunks na ordem correta.



O coordenador busca os chunks sequencialmente, na ordem definida pelo `chunk_id`, e os concatena para reconstruir o arquivo original. Esta é uma decisão consciente de simplicidade.

EVIDÊNCIA: DISTRIBUIÇÃO REAL DO CLUSTER

Saída do coordenador durante um PUT de um arquivo de 200 KB (4 chunks de 64 KB).

```
bash - 1000x600
```

```
$ python run_cli.py put video.mp4 docs/video.mp4
```

```
[PUT] path=docs/video.mp4 chunk=0 primary_node=node3 primary_shard=3 size=65536
```

```
[COORDENADOR] tentativa 1/3 | op=PUT path=.chunks/... node=node3 shard=3
```

```
[PUT] path=docs/video.mp4 chunk=1 primary_node=node1 primary_shard=1 size=65536
```

```
[COORDENADOR] tentativa 1/3 | op=PUT path=.chunks/... node=node1 shard=1
```

```
[PUT] path=docs/video.mp4 chunk=2 primary_node=node2 primary_shard=2 size=65536
```

```
[COORDENADOR] tentativa 1/3 | op=PUT path=.chunks/... node=node2 shard=2
```

```
[PUT] path=docs/video.mp4 chunk=3 primary_node=node1 primary_shard=1 size=8192
```

```
✓ Arquivo salvo com 4 chunk(s).
```

```
Distribuição: node1:2, node2:1, node3:1
```

Cada chunk é roteado de forma independente. Para um único arquivo de 200 KB, a carga foi distribuída entre os três nós do cluster, comprovando o balanceamento determinístico.

ENGENHARIA DE SISTEMAS: ESCOLHAS E CONSEQUÊNCIAS

PROTOCOLO DE TRANSPORTE [COMM_STACK]

[X] TCP PURO + PROTOBUF

Controle total sobre o framing, menor overhead e maior valor didático para entender a pilha de rede.

[] gRPC / HTTP2

Abstração excessiva para o escopo do projeto e overhead de cabeçalhos desnecessários para chunks binários.

ALGORITMO DE DISTRIBUIÇÃO [SHARDING_STRAT]

[X] HASH MODULAR (SHA-256)

Simplicidade de implementação e eficácia garantida para clusters com número fixo de nós.

[] CONSISTENT HASHING

Melhor para elasticidade (nós entrando/saindo), mas adiciona complexidade que faz mais sentido no Marco 4.

ARMAZENAMENTO DE METADADOS [STATE_MGMT]

[X] PERSISTÊNCIA EM JSON

Inspeção humana fácil e simplicidade de implementação sem dependências externas.

[] BANCO DE DADOS (SQL/KV)

Oferece transações robustas, mas introduz complexidade de infraestrutura prematura para esta fase.

O QUE O MARCO 2 (AINDA) NÃO FAZ

Limitações conscientes que delimitam o escopo do marco e serão endereçadas em iterações futuras.

KNOWN_ISSUE

SEM REPLICAÇÃO

Cada chunk possui apenas uma cópia física. Se um nó de armazenamento falhar permanentemente, os dados contidos nele tornam-se inacessíveis.

KNOWN_ISSUE

SEM HEARTBEAT

O coordenador assume que todos os nós registrados estão operacionais. Não há detecção em tempo real de quedas de nós ou latência excessiva de rede.

KNOWN_ISSUE

SPOF (COORDINATOR)

O Coordenador é um ponto único de falha. Se o processo central cair, todo o cluster fica indisponível para novas operações e consultas.

KNOWN_ISSUE

PUT SEM ROLLBACK

Se um chunk falhar no meio do processo, podem ficar chunks órfãos nos nós. A operação retorna erro mas não desfaz as gravações parciais.

ROADMAP: RUMO À RESILIÊNCIA TOTAL

[PHASE_03]

MARCO 3: RESILIÊNCIA

- Replicação de chunks (fator N)
- Detecção de falhas via Heartbeat
- Recuperação automática de dados
- Política de consistência (eventual ou forte)

[PHASE_04]

MARCO 4: ELASTICIDADE

- Adição e remoção dinâmica de nós
- Rebalanceamento automático de carga
- Hashing consistente
- Otimização de tráfego de rede

[PHASE_05]

MARCO 5: OPERAÇÃO

- Testes de carga automatizados
- Coleta de métricas (latência, throughput)
- Análise de gargalos e escalabilidade
- Alta disponibilidade do coordenador

```
$ logout --reason="presentation_complete"
```

DFS: Sistema de Arquivos Distribuído

Dúvidas?

AUTHORS: Higor Ferreira Silva
Vitória Mendonça

PROJECT: DFS Marco 2

SESSION: Closed Successfully

```
$ exit █
```